

CS & IT ENGINEERING



Data Structure & Programming

Linked Lists

Lecture No.- 01



By- Satya sir

Recap of Previous Lecture



- Arrays

- 1-D array

- 2-D array

- 3-D array

Initialization/
Assignment

- Address of an Element with random index

- Searching an Element in an array

- Linear Search

- Binary Search

Topics to be Covered



- Array Insertion
 - Array Deletion
 - Linked Lists
 - Characteristic
 - Types
 - Construction
 - Insertion
- SLL



Topic : 1-D Arrays – ISRO 2009



A one dimensional array A has indices 1....75. Each element is a string and takes up three memory words. The array is stored at location 1120 decimal. The starting address of A[49] is _____

$$B = 1120 \quad \text{Starting index, } x = 1, \quad \text{Size, } n = 3$$

$$\{ A[i] = B + (i - x) * n$$

$$\{ A[49] = 1120 + (49 - 1) * 3$$

$$= 1120 + 48 * 3$$

$$= 1120 + 144$$

$$= \underline{\underline{1264}}$$



Let A be an array containing integer values. The distance of A is defined as the minimum number of elements in A that must be replaced with another integer so that the resulting array is sorted in a non-decreasing order. The distance of the array [2, 5, 3, 1, 4, 2, 6] is 3

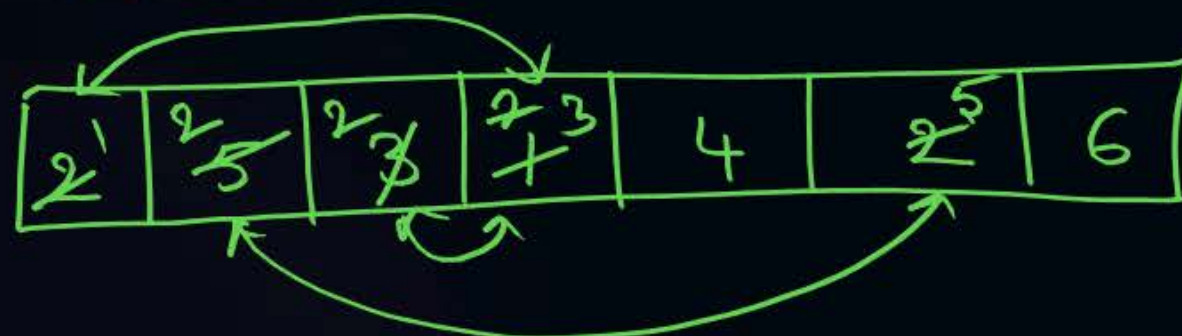
length of given array = 7

Length of LIS = 4

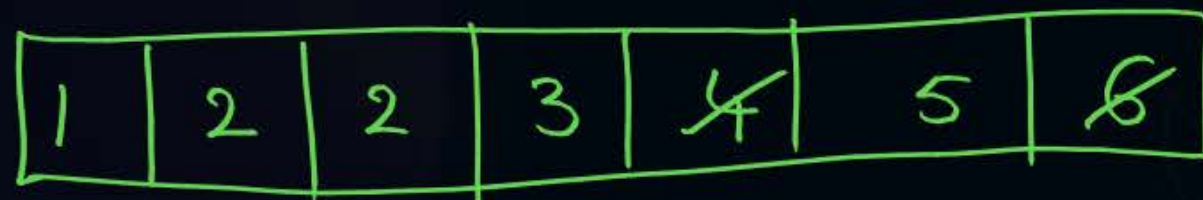
$\Rightarrow (7 - 4) = 3$ Elements need to be sorted.

length

Given Array



Result Array



Longest Increasing Sub Sequence

[2, 3, 4, 6]

✓ 2 5 3 1 4 2 6 ✓
IS: [2 5 6] length = 3
IS: [5 6] length = 2

IS: [1 4 6] 3

IS: [2, 3, 4, 6] 4

IS: [1, 2, 6] 3

IS: [3, 4, 6] 3

IS: [4, 6] 2

⋮



Topic : 1-D Arrays – ISRO 2018



An array A consists of n integers in locations A[0], A[1]A[n-1]. It is required to shift the elements of the array cyclically to the left by k places, where $1 \leq k \leq (n-1)$. An incomplete algorithm for doing this in linear time, without using another array is given below. Complete the algorithm by filling in the blanks.

Assume all the variables are suitably declared.

min = n; i = 0;

while ($i < \min$) {

temp = A[i]; j = i;

while ($j \neq (n+i-k) \% n$) {

A[j] = $A[(j+k) \% n]$

j = (j + k) mod n;

If (j < min) then

min = j;

}

A[(n + i - k) mod n] = _____

i = $i+1$

K = 2

Result

5	7	9	1	3
---	---	---	---	---

Let

0	1	2	3	4
1	3	8	7	9

i = 0 temp = 1 j = 0

0 % 3 = 0 A[0] = A[2]

j = 2 2 < 5 True min = 2

2 % 3 = 3 True

A[2] = A[4]

j = 4

4 % 3 = 1 True

A[4] = A[1]

j = 1

~~A. i > min; j != (n+i) mod n; A[j + k]; temp; i + 1;~~

~~B. i < min; j != (n+i) mod n; A[j + k]; temp; i + 1;~~

~~C. i > min; j != (n+i+k) mod n; A[(j + k)]; temp; i + 1;~~

D. i < min; j != (n+i-k) mod n; A[(j + k) mod n]; temp; i + 1;

1 % 3 = 3 True

A[1] = A[3]

j = 3

3 % 3 = 3 False

A[3] = 1

i = 2

2 < 2 False



Topic : Arrays Insertion, Deletion



Array - Insertion / Add an Element

Let $\text{Array}[10] = \{1, 3, 5, 7\}$ $\text{count} = 4$

Worst-case : Insert at first Position



Value = 8 at First Position = index 0

$i = \text{count} // 6$

while ($i > 0$)

{ $\text{Array}[i] = \text{Array}[i-1]$

$i = i - 1$

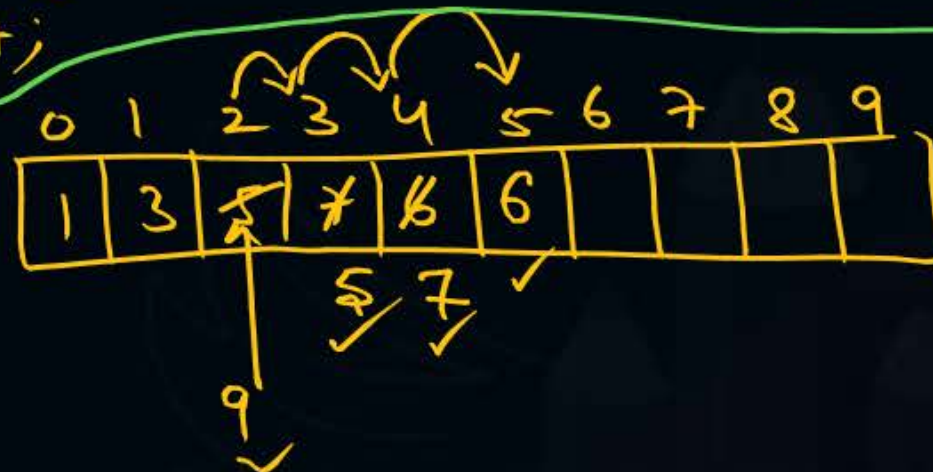
} $\text{Array}[i] = \text{value}; \text{count}++$

$\left. \begin{array}{l} \text{ntimes} \\ \Rightarrow \text{Time Complexity} \\ \underline{\underline{O(n)}} \end{array} \right\}$

Best case: Insert at End of list
Let $\text{value} = 6$
 $\text{Array}[\text{count}] = \text{value};$
 $\text{count}++;$
Time Complexity = $\underline{\underline{O(1)}}$

Average case: Insert at middle of list

Array



Position = 2, Value = 9

$i = \text{count} // i = 5$

while ($i > \text{Position}$)

{ $\text{Array}[i] = \text{Array}[i-1]$

$i = i - 1$

} $\text{Array}[\text{Pos}] = \text{Value}; \text{count}++;$

$n/2 \text{ times} \Rightarrow \text{Time Complexity: } O\left(\frac{n}{2}\right) \Rightarrow O\left(\frac{1}{2} * n\right) \Rightarrow \underline{\underline{O(n)}}$



Deletion from Array / Remove an Element

Best case : Last Element

Let $A = [1, 3, 5, 7, 9]$ count = 5

Printf("Removed Element is %d", $A[--\text{count}]$); // Time Complexity = $O(1)$

Average case : Deletion of middle element : Time Complexity : $O(\frac{n}{2}) \Rightarrow \underline{O(n)}$

$\Rightarrow$ shift Elements right of middle to left

$A[i] = A[i+1]$

Worst-Case : Delete first element : Shift all elements left by 1 Position $\Rightarrow$ Time Complexity : $O(n)$



Topic : SLL – Creation, Insertion



2-D Arrays – LTM, UTM

$n = \text{No. of rows / cols} = 3$ [Square Matrix]

Let $x[3][3] = \{1, 3, 5, 7, 9, 12, 16, 18, 2\}$

	0	1	2
0	1	3	5
1	7	9	12
2	16	18	2

UTM (Upper Triangular Matrix) is the part above the diagonal (yellow outline).

LTM (Lower Triangular Matrix) is the part below the diagonal (orange outline).

LTM

1	0	0
7	9	0
16	18	2

UTM

1	3	5
0	9	12
0	0	2

No. of Elements = 6

= 6

$$\Rightarrow \frac{n(n+1)}{2}$$

$n = \text{No. of rows (or) cols in Square Matrix}$



Linked Lists

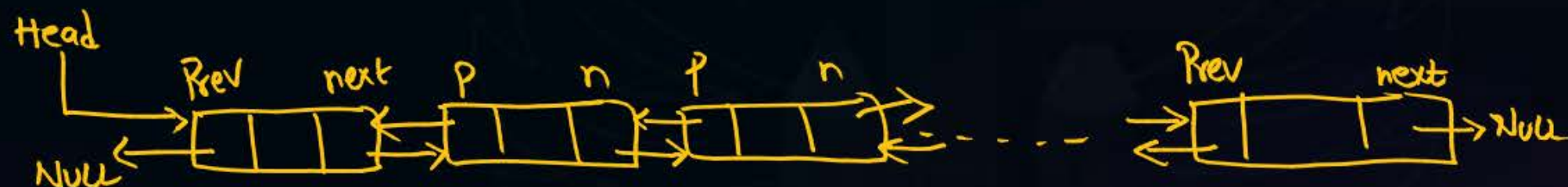
- List (or) group of elements that are linked together.

- 3 Types of Linked Lists :

1) Singly Linked List (SLL)



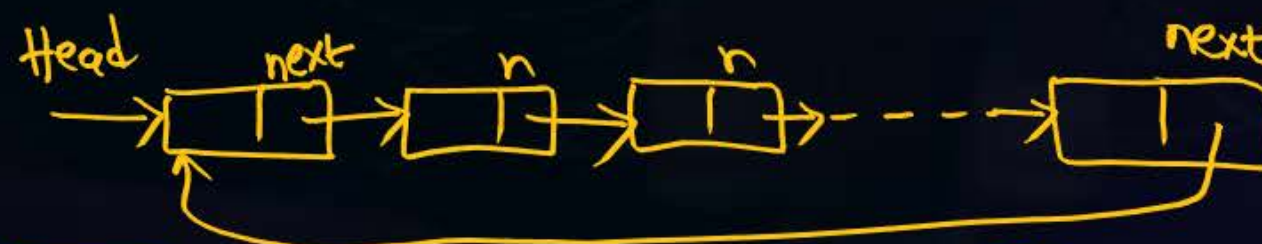
2) Doubly Linked List (DLL)



3) Circular Linked List (CLL)

└ Circular Singly Linked List (default)

└ Circular doubly Linked List





Topic : SLL – Creation, Insertion



- Create Node
- Insert Node(s)
- Construct Linked List
- Delete an element
- Search for an element
- Access (or) Traverse LL

To be discussed...



2 mins Summary



- H/W Questions Solving
- Array Insertion, Deletion
- Array - LTM, UTM
- Linked Lists Introduction

To be contd



THANK - YOU